

OPERATING SYSTEMS - BCSE303L

SLOT : A2+TA2

TEAM MEMBER – DETAILS

MEDARAMETLA LIKHITH – 24BYB0054.

KURRA LEELA KRISHNA – 24BYB0004.

PRASANNA MANIKANDAN – 24BYB0060.

OSentinel: Intelligent Defense at the Operating System Level

Abstract

OSentinel is an intelligent host-based security framework for Ubuntu Linux that combines rule-based detection with machine learning to identify suspicious and abnormal activity in real time. It monitors processes, file system activity, and network connections, converting system events into useful behavioral features.

Two detection engines work together the rule engine identifies known threats such as suspicious program execution or mass file encryption, while the machine-learning model detects unusual behavior that may indicate new or unknown threats. Their results are combined into a risk score from 0 to 100 comparing with a pre-defined threshold which logging low-risk events to alert users and applying controlled procedures for high-risk activity.

A web dashboard displays system activity, security incidents, and risk trends in one place. The modular architecture separates data collection, feature extraction, detection, risk assessment, and visualization, making the system easier to expand in the future of OSentinel into a multi-OS security solution.

Keywords

Signature-based detection, Machine Learning Security, Isolation Forest, Random Forest, Risk Scoring, Ransomware Detection, Web-Based Security Dashboard.

Introduction

Cybersecurity threats are becoming more advanced, with attackers increasingly using fileless attacks and ransomwares that can initially appear normal. Traditional security systems mainly rely on known signatures, so they may miss new or modified threats**.** OSentinel addresses this gap by focusing on how a system behaves rather than only looking for known malicious files.

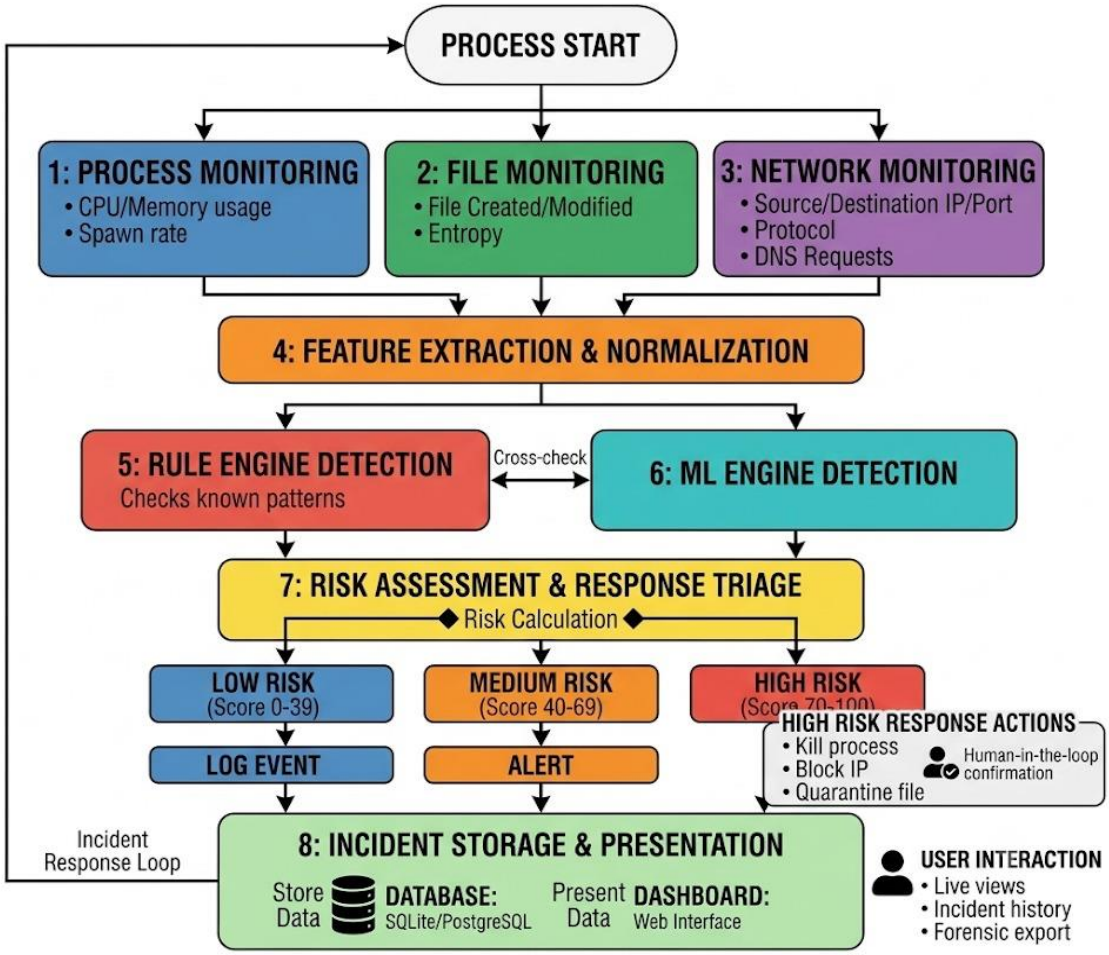
OSentinel monitors three key areas of an Ubuntu system processes, file activity, and network connections. The collected events are converted into useful features such as file-writing rates, connection frequency, and file entropy, which can help identify ransomware-like activity. A rule engine detects known

suspicious patterns, while a two-stage machine-learning pipeline identifies unusual behavior and potential unknown threats.

The results from both detection methods are combined into a risk score from 0 to 100**,** which determines the appropriate response, such as logging, raising an alert, or applying controlled and reversible containment. A web dashboard provides a real-time view of processes, incidents, and the reasons behind each risk score, keeping the system transparent and easier for an operator to monitor.

Based on the risk level, the system can simply record normal activity, generate alerts and incidents for suspicious behavior, or perform controlled and reversible containment for high-risk activity. A web-based dashboard provides a real-time view of processes, network activity, security incidents, and risk scores, while also showing the evidence behind each detection. This makes the system easier to monitor and keeps automated decisions transparent to the user.

Initially Implemented on Linux Ubuntu based platform for easier feature extraction and later it is extended to the multi OS.



Literature Review

1.**Host Data," ACM Computing Surveys.**

This survey covers different host-based intrusion detection systems and the types of data they use, such as system logs, audit data, file systems, and system calls. It also discusses the datasets available for security research and their limitations. This work helped guide OSentinel's decision to monitor three main areas: processes, files, and network activity. It also supports the idea of using both public datasets and data collected directly from the system instead of depending on only one source.

2. "Deep Learning-based Intrusion Detection Systems: A Survey," arXiv, 2025.

This survey looks at different machine-learning methods used for intrusion detection. It compares traditional machine-learning models with newer deep-learning methods that use relationships between system activities. The study shows that tree-based models are a practical choice when working with simple, table-based data like the features collected by OSentinel. This supports OSentinel's decision to use lightweight machine-learning models instead of more complex deep-learning models.

3. "Machine learning-enabled hybrid intrusion detection system with host data transformation and an advanced two-stage classifier," ScienceDirect, 2024.

This paper explains that network-based detection alone may miss attacks that mainly affect the host, such as attacks involving files and system activity. It combines host and network information and uses two stages for detection. This is similar to OSentinel's approach, where the first stage looks for unusual behavior and the second stage identifies the possible type of threat using information from processes, files, and network activity.

4. "Host-Based Network Intrusion Detection via Feature Flattening and Two-stage Collaborative Classifier," arXiv, 2023.

This work explains how complex host data can be converted into simple feature values that machine-learning models can understand. It then uses two classification stages to identify attacks. This approach is useful for OSentinel because Module 4 also needs to convert raw process, file, and network events into fixed feature values before sending them to the rule engine and ML models.

5. Negandhi, Trivedi et al., "Intrusion Detection System Using Random Forest on the NSL-KDD Dataset," Springer, 2019.

This paper uses Random Forest to detect network attacks using the NSL-KDD dataset. It also selects the most useful features to make the model faster and more accurate. The study shows that Random Forest performed better than several other common models in their tests. This supports OSentinel's choice of Random Forest for its supervised classification stage and the use of feature selection to improve performance.

6. "Intrusion detection system combined enhanced random forest with SMOTE algorithm," EURASIP Journal on Advances in Signal Processing, 2022.

This paper focuses on the problem of having much more normal data than attack data. It uses Random Forest along with SMOTE to create additional samples for the smaller attack group. This is useful for OSentinel because its own collected data is also expected to contain many more normal activities than attack activities. The same approach can therefore be considered when preparing the training data for Module 6.

7. "Anomaly Detection in Computer Networks Using Isolation Forest in Data Mining," ResearchGate, 2025.

This paper compares Isolation Forest with other methods such as One-Class SVM and Local Outlier Factor. The study reports that Isolation Forest provided good detection results, fewer false alarms, and required less computing power. This supports OSentinel's choice of Isolation Forest for finding unusual activity, especially because the security agent needs to work continuously without using too many system resources.

8. "Comparing Autoencoder and Isolation Forest in Network Anomaly Detection,"ResearchGate,2023.

This study compares Isolation Forest with a deep-learning autoencoder for detecting unusual network activity. The results show that Isolation Forest can work well with table-based security data while requiring less training data and computing power than more complex deep-learning methods. This supports OSentinel's decision to keep its main ML system simple and lightweight.

9. Lee et al., "Machine Learning Based File Entropy Analysis for Ransomware Detection in Backup Systems,"IEEE, 2019.

This paper uses file entropy along with machine learning to identify files affected by ransomware. Encrypted files usually have high entropy compared with normal files. This idea is directly used in OSentinel through its file-entropy feature. By checking for a large number of files being changed and showing high entropy, OSentinel can identify possible ransomware activity at an early stage.

10. "eBPF-PATROL: Protective Agent for Threat Recognition and Overreach Limitation using eBPF in Containerized and Virtualized Environments,"2025.

This paper looks at how eBPF-based security tools can collect system activity directly from the Linux kernel with low performance impact. It also points out that many existing tools mainly depend on fixed rules. This gives OSentinel a possible future improvement: using eBPF for faster and deeper system monitoring instead of depending only on tools such as psutil OSentinel could then combine this improved monitoring with its rule-based and machine-learning detection system.

11. S. Forrest, S. A. Hofmeyr, A. Somayaji, T. A. Longstaff, "A Sense of Self for Unix Processes," IEEE Symposium on Security and Privacy, 1996.

This paper proposes that short sequences of system calls made by a running process form a stable pattern of normal behavior, which the authors call the "self" of the process. Programs that behave differently from this pattern, for example because of an attack, can be detected as anomalies. This is one of the earliest works on behavior-based host intrusion detection, and it forms the basis for OSentinel's idea of watching how processes behave instead of only matching known signatures.

12. S. A. Hofmeyr, S. Forrest, A. Somayaji, "Intrusion Detection using Sequences of System Calls," Journal of Computer Security, 6(3), 1998.

This paper extends the earlier work by building profiles of normal system-call sequences and measuring how far new behavior deviates from them. The results show that several real attacks on privileged programs can be detected with a low false-alarm rate. This supports the use of anomaly-based detection in OSentinel's process monitoring, where normal behavior is learned and unusual behavior raises the risk score.

13. G. Creech, J. Hu, "A Semantic Approach to Host-Based Intrusion Detection Systems Using Contiguous and Discontiguous System Call Patterns," IEEE Transactions on Computers, 63(4), 2014.

This work builds meaningful features from patterns of system calls, both contiguous and discontiguous, and uses them to train a classifier for host-based detection. It is evaluated on the ADFA-LD dataset and reports high detection rates with few false alarms. This is relevant to OSentinel because Module 4 also converts raw host events into meaningful features, and ADFA-LD is a possible public dataset for host-based testing.

14. F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation Forest," IEEE International Conference on Data Mining (ICDM), 2008.

This is the original paper that introduced the Isolation Forest algorithm. Instead of modeling normal data, it isolates anomalies using random splits, and unusual points need fewer splits to be separated. The method has linear time complexity and low memory use. This is the core algorithm behind OSentinel's unsupervised stage for detecting unknown or unusual behavior.

15. F. T. Liu, K. M. Ting, Z.-H. Zhou, "Isolation-Based Anomaly Detection," ACM Transactions on Knowledge Discovery from Data, 6(1), 2012.

This journal paper gives a more detailed explanation and evaluation of isolation-based anomaly detection. It shows that the method performs better than several distance-based and density-based approaches in both accuracy and speed. This gives further support for choosing Isolation Forest over methods such as Local Outlier Factor or One-Class SVM in OSentinel.

16. V. Chandola, A. Banerjee, V. Kumar, "Anomaly Detection: A Survey," ACM Computing Surveys, 41(3), 2009.

This survey gives a broad overview of anomaly-detection techniques and where they are used. It groups the methods into categories such as statistical, nearest-neighbor, and classification-

based, and explains the strengths and weaknesses of each. It provides the theoretical background for the anomaly-detection stage of OSentinel and helps justify the choice of method.

17. Y. Mirsky, T. Doitshman, Y. Elovici, A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," NDSS, 2018.

Kitsune is a lightweight system that detects network attacks in real time using a group of small autoencoders, without needing labeled attack data. It is designed to run online on modest hardware. This shows that unsupervised, real-time detection with low overhead is practical, which is useful for OSentinel's network monitoring component.

18. L. Breiman, "Random Forests," Machine Learning, 45(1), 2001.

This is the original paper on Random Forest. It builds many randomized decision trees and combines their votes, which makes the model accurate, resistant to overfitting, and able to show which features are most important. This is the basis for the supervised classification stage used in OSentinel.

19. N. V. Chawla, K. W. Bowyer, L. O. Hall, W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," Journal of Artificial Intelligence Research, 16, 2002.

This paper introduces SMOTE, which creates synthetic samples of the minority class instead of simply copying existing ones. It shows that this improves classifier performance when one class is much rarer than the other. Since OSentinel's data will contain far more normal events than attack events, SMOTE is a suitable method for balancing the training data.

20. M. Tavallaee, E. Bagheri, W. Lu, A. A. Ghorbani, "A Detailed Analysis of the KDD CUP 99 Data Set," IEEE Symposium on Computational Intelligence for Security and Defense Applications, 2009.

This paper analyzes problems in the KDD CUP 99 dataset, especially the large number of duplicate records that bias the results, and proposes the cleaned NSL-KDD dataset. It explains where NSL-KDD comes from and why results on it should be read carefully. This motivates OSentinel's plan to also use data collected from its own host environment.

21. I. Sharafaldin, A. H. Lashkari, A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," ICISSP, 2018.

This paper presents the CICIDS2017 dataset, which contains realistic benign traffic and modern attacks along with labeled flow-based features. It was created to overcome the weaknesses of older datasets. It is a possible public dataset for training and validating the features used in OSentinel's network module.

22. M. Ring, S. Wunderlich, D. Scheuring, D. Landes, A. Hotho, "A Survey of Network-Based Intrusion Detection Data Sets," Computers & Security, 86, 2019.

This survey compares many public intrusion detection datasets and describes their properties and weaknesses. It notes that many are outdated, imbalanced, or not realistic, and it suggests

criteria for choosing a suitable dataset. This supports OSentinel's approach of combining public datasets with data collected directly from the system.

23. A. Kharaz, S. Arshad, C. Mulliner, W. Robertson, E. Kirda, "UNVEIL: A Large-Scale, Automated Approach to Detecting Ransomware," USENIX Security Symposium, 2016.

UNVEIL runs ransomware in a controlled environment and detects it from its file-system behavior, such as the way files are read, overwritten, and encrypted. It reports a high detection rate with very few false alarms. This supports the file-activity rules in OSentinel that look for mass file modification and encryption-like behavior.

24. N. Scaife, H. Carter, P. Traynor, K. R. B. Butler, "CryptoLock (and Drop It): Stopping Ransomware Attacks on User Data," IEEE International Conference on Distributed Computing Systems (ICDCS), 2016.

CryptoLock detects ransomware early by combining several file-change indicators, including changes in file entropy, file type, and similarity to the original file, along with mass deletion. It can stop an attack after only a small number of files are affected. This is very close to OSentinel's use of entropy and write-rate features and its goal of early, controlled containment.

25. A. Continella et al., "ShieldFS: A Self-healing, Ransomware-aware Filesystem," ACSAC, 2016.

ShieldFS monitors file-system activity, uses models of normal and ransomware behavior to detect attacks, and can restore files that were modified before detection. It does this with low performance overhead. This supports OSentinel's idea of reversible containment for high-risk activity.

26. R. Berrueta, D. Morato, E. Magaña, M. Izal, "A Survey on Detection Techniques for Cryptographic Ransomware," IEEE Access, 7, 2019.

This survey reviews ransomware detection methods based on file activity, network behavior, and entropy, and discusses the strengths and limits of each. It shows that using more than one source of evidence gives better results. This justifies OSentinel's design of combining process, file, and network monitoring.

27. A. L. Buczak, E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Communications Surveys & Tutorials, 18(2), 2016.

This survey describes many machine-learning and data-mining methods used for intrusion detection, along with their complexity and data requirements. It helps in comparing methods and choosing ones that fit the available data. It gives overall support for using machine learning together with rule-based detection in OSentinel.

28. R. Sommer, V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," IEEE Symposium on Security and Privacy, 2010.

This paper explains why machine-learning anomaly detection is hard to use in real systems. The main reasons are the high cost of false alarms and the lack of realistic training data. It is useful for OSentinel because it supports combining rules with machine learning, using a risk score with thresholds, and showing the reasons behind each alert.

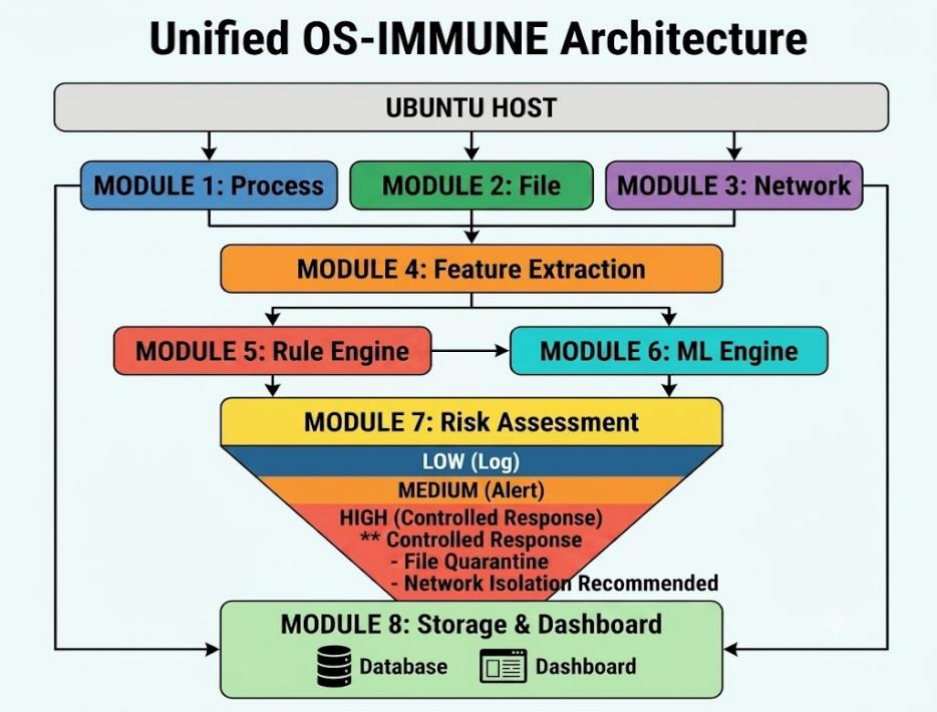
29. S. M. Lundberg, S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," NeurIPS, 2017.

This paper introduces SHAP, a method that explains a model's prediction by showing how much each feature contributed to it. It works with many types of models, including tree-based ones. This could be used in OSentinel's dashboard to show the evidence behind each risk score and keep automated decisions transparent.

30. D. Vieira et al., "Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications," ACM Computing Surveys, 53(1), 2020.

This survey explains eBPF and XDP, which allow programs to run safely inside the Linux kernel for fast monitoring and processing. It covers the concepts, challenges, and applications. This provides the technical background for OSentinel's planned future move to eBPF-based data collection.

System Architecture



Tech Stack

Data collection: auditd , pyinotify , psutil, netstat
Backend: Python (Flask/FastAPI) orchestration
ML: scikit-learn (Isolation Forest, Random Forest)
Storage: SQLite/PostgreSQL for structured logs.
Dashboard: React/Vue frontend WebSocket Events for live updates

Conclusion:

OSentinel is designed to provide better security by watching how the system behaves instead of depending only on known attack signatures. It monitors processes, files, and network activity and uses both rules and machine learning to find suspicious behavior.

The project combines Isolation Forest, Random Forest, and risk score comparing to pre defined thresholds identify unusual activities and decide how serious a threat is. Based on the risk level, the system can log the activity, generate an alert, or take a controlled action. The web dashboard also makes it easier to view system activity, alerts, and incidents in one place.

The modular design makes OSentinel easier to develop, test, and improve because each module has a separate purpose. For the first stage, the system is being developed on Ubuntu Linux which provides a suitable environment for collecting and analyzing system activity. the same idea is extended to Windows and macOS, making OSentinel a single security solution for different operating systems.